

Self-Modifying Neural Network (Hybrid Adaptive System)

1. Problem Statement

Modern neural networks use fixed architectures designed by humans. While weights are learned, the structure itself cannot adapt. This limits flexibility, efficiency, and adaptability.

We aim to build a system that can dynamically change its structure during learning, similar to biological intelligence.

2. Objective

Develop a neural network that can learn weights, adjust neuron importance using alpha parameters, prune unnecessary neurons, and grow when needed to match task complexity.

3. Challenges and Solutions

Catastrophic Forgetting

Important knowledge may be lost when neurons are pruned. Solution: Use importance scoring and protect critical neurons.

Structural Credit Assignment

It is unclear what should change when errors occur. Solution: Split responsibility into levels: weights, alpha, and structure.

Bloat Problem

Network may grow excessively and overfit. Solution: Add cost of existence using L1 penalty on alpha.

Search Space Explosion

Too many possible architectures exist. Solution: Use continuous relaxation instead of discrete search.

Knowledge Transfer During Mutation

Structure changes can destroy learned knowledge. Solution: Use smooth alpha-based evolution instead of abrupt changes.

Interpretability Problem

System may become a black box. Solution: Track alpha values, neuron count, and performance over time.

4. Requirements

Technical Requirements:

- 1 Python programming
- 2 PyTorch framework
- 3 NumPy and Matplotlib libraries

Mathematical Requirements:

- 1 Linear Algebra (vectors, matrices)
- 2 Calculus (gradients, optimization)
- 3 Basic probability and loss functions

Conceptual Requirements:

- 1 Understanding of neural networks
- 2 Backpropagation
- 3 Optimization techniques

5. Final Outcome

A self-adaptive neural network that can dynamically grow, shrink, and optimize its structure while preserving important knowledge. The system becomes efficient, flexible, and closer to biological intelligence.